

Learning for Next Time

Turning painful support incidents into guardrails that stop the same mistake from happening again.

The idea in one line

When a support problem gets resolved, the workspace saves one guardrail built from what just happened. Later, if you are about to make the same mistake in the console, it warns you **before** you commit the change. The lesson protects you on its own, so nobody has to remember it.

The problem it solves

Today, help articles sit in a side panel or on a different website you read later. A tip you have to remember is a tip you forget. So the same mistakes keep happening, and support keeps solving the same problems.

How it works

There are two parts:

- **The memory (a guardrail database).** Each entry stores what to watch for, what went wrong last time, the fix, who it applies to, and how badly it hurt.
- **The watcher (the bot).** As you work in the console, it turns your action into a plain intent ("making this bucket public") and checks it against the memory. On a match, it warns you right there.

The check happens **before** the change is saved, like a quick pre-flight check, so the mistake is caught rather than just reported afterward.

The loop

incident happens → resolved in the workspace → a guardrail is written to memory → later you act in the console → the watcher matches it → it warns you before you commit → mistake avoided.

Key design decisions

- **It acts, it doesn't just advise.** The value is a one-click "pin this fix and watch for it" button. The text is only the explanation; the button is the product.
- **Calm by design.** The tip appears once, only after the problem is resolved, as an optional note the user chooses to open. Never a popup during a crisis.
- **Two layers.** A shared baseline of common AWS footguns everyone gets, plus a personal layer built from the customer's own history. The personal layer is the part nobody can copy.
- **Two kinds of matching.** Exact rules for clear-cut danger (open ports, deleting a database with no backup), and fuzzy matching for the gray area ("this looks like that thing that bit you before").

- **Guarding against false alarms.** A 24-hour mute is the default, so it goes quiet while you work but comes back on its own. Every dismissal is also a signal that trains the system to stop showing noisy guardrails.

What's in scope (and what isn't)

Not every small console error, and not only formal support cases. The right filter is two things at once: **did it actually hurt**, and **can it happen again the same way**. High impact plus repeatable is the sweet spot.

- In scope: the EKS crash from a changed health-check setting; a bucket made public; an over-wide IAM policy; a database deleted with no snapshot; a surprise bill from a forgotten instance.
- Out of scope: a typo fixed five seconds later; a one-time AWS-side outage with nothing to guard against.

Building it for the demo

You cannot add features to the real AWS console (it is Amazon's production frontend, with no plugin system). The right approach is a self-built mock of the one console screen the scenario touches, with the feature inside it. This matches the brief's ask for a medium-fidelity, clickable prototype.

- Build only the one screen the scenario needs (the deploy/settings page).
- Fake everything behind it: no real database or matching bot. Hardcode the one scenario so changing the setting triggers the warning.
- Keep to the single EKS story, then show two or three other guardrails as a "your safety net so far" list to suggest breadth.

Why it's novel

The clever part isn't the enforcement — AWS already has pieces for that. It's that the guardrails **write themselves from the customer's own support history** and get matched by an agent in real time. Every scar becomes a protection, so the safety net grows the more you use support.